

Compte-Rendu de Projet

Serveur IA Local & Detection de Plagiat

Projet	SAE204 — Soutenance Cybersecurite
Formation	BUT Reseaux & Telecommunications — Auxerre
Auteur	Enzo Maresia
Depot	github.com/AYK0-prog/Local_AI_server
Date	Juin 2026
Statut	Phases 0, 1, 2 terminees — Phase 3 en cours

Sommaire

1. Présentation du projet
2. Architecture reseau et materielle
3. Identifiants et controle d'accès
4. Phase 0 — Environnement logiciel et deploiement
5. Phase 1 — Pipeline de detection de plagiat (CLI)
6. Phase 2 — Application web Flask
7. Verification externe — Etat actuel
8. Securite et conformite
9. Deroulement du projet — Seances de travail
10. Inventaire des fichiers de projet
11. Feuille de route — Phase 3
12. Glossaire
13. Recapitulatif des ports et chemins

1. Presentation du projet

1.1 Contexte

Ce projet repond a une demande du departement R&T d'Auxerre : disposer d'un **serveur d'Intelligence Artificielle 100 % local**, independant des services cloud commerciaux (ChatGPT, Gemini, etc.), pour deux usages distincts. L'angle directeur est la **confidentialite des donnees** : tous les traitements sont effectues en local, sans envoi de donnees sensibles vers des serveurs tiers. Le pipeline est desormais **100 % local**, sans aucune dependance externe.

1.2 Objectifs

Objectif	Description	Livrable
A	Heberger une IA locale, legere, rapide et autonome	Docker + Ollama + OpenWebUI
B	Detecter la fraude/plagiat dans les comptes-rendus de TP	Pipeline Python (CLI) + Flask

1.3 Cahier des charges

- Serveur capable d'heberger une intelligence artificielle
- IA servie par Ollama (API REST locale, port 11434)
- IA legere, rapide, efficace, autonome et portant un nom (assistant base sur Qwen)
- Systeme de detection de fraude en Python (pipeline multi-signaux)
- Utilisation headless via interface web avec interaction vocale
- Compatibilite Home Assistant — realise et presente comme bonus

1.4 Stack technologique retenue

Apres evaluation de LiteLLM, LM Studio et OpenCode, la stack retenue est volontairement minimale et entierement auto-hebergée. LM Studio a ete rejete : le Xeon E5-2630 v3 ne dispose pas du jeu d'instructions **AVX2** requis. LiteLLM a ete installe mais la liaison avec Ollama n'a jamais ete fonctionnelle.

Couche	Technologie	Role
Systeme	Linux Mint 22.3	Base headless legere
Conteneurisation	Docker Engine (CLI)	Isolation des services IA
Serveur de modeles	Ollama (API REST :11434)	Chargement et inference des LLM
Interface web IA	OpenWebUI (port 3000)	Front conversationnel — acces admin
Detection plagiat	Python 3.12.3 + Flask	Pipeline d'analyse + interface web
Second avis IA	Ollama local — llama3.1:8b	Second verdict 100 % local

1.5 Etat d'avancement

Phase	Description	Statut
Phase 0	Environnement (Docker + Ollama + OpenWebUI + reseau)	[OK] Terminee
Phase 1	Script CLI de detection de plagiat	[OK] Testee en production
Phase 2	Application web Flask	[OK] Fonctionnelle
Phase 3	Preparation soutenance	[EN COURS]

2. Architecture reseau et materielle

2.1 Topologie actuelle

Le serveur IA est un **PC custom connecte directement en filaire** au reseau prive de la salle reseaux de l'IUT. Ce reseau prive dispose d'un acces Internet. Aucune passerelle intermediaire n'est configuree : les clients (Lenovo, MacBook) accedent aux services directement via ce reseau.

```
Reseau prive salle reseaux IUT (172.17.0.0/xx)
```

```
Acces Internet disponible – connexion filaire
```

```
|
```

```
+-----+-----+
```

```
| SERVEUR IA |
```

```
| PC Custom |
```

```
| 2x Xeon E5-2630 v3 |
```

```
| RTX 3060 Ti – 8 Go VRAM |
```

```
| 32 Go DDR3 |
```

```
| Linux Mint 22.3 (headless) |
```

```
| IP : 172.17.0.x (DHCP) |
```

```
| Ollama / OpenWebUI / Flask |
```

```
+-----+-----+
```

```
|
```

```
+-----+-----+
```

```
| |
```

```
+-----+ +-----+
```

```
| Lenovo | | MacBook |
```

```
| Client | | Client |
```

```
+-----+ +-----+
```

2.2 Fiche technique du serveur

Composant	Detail
Processeurs	2x Intel Xeon E5-2630 v3 (8 coeurs / 16 threads chacun — 32 threads au total)
GPU	NVIDIA RTX 3060 Ti — 8 Go VRAM
RAM	32 Go DDR3 (barrettes defectueuses retirees en seance 3)
Systeme	Linux Mint 22.3 — mode headless
Python	3.12.3
Utilisateur	enzo
Reseau	Filaire — salle reseaux IUT — IP 172.17.0.x (DHCP)

2.3 Note historique

En debut de projet (seances 1 a 5), le reseau reposait sur un sous-reseau isole **10.42.0.0/24** cree via un switch physique, avec un Lenovo Ideapad en passerelle NAT et un Raspberry Pi 5 hebergeant Home Assistant OS a l'IP fixe 10.42.0.100. Cette infrastructure a ete simplifiee : le serveur est desormais raccorde directement au reseau de la salle reseaux de l'IUT.

2.4 Note sur Home Assistant (bonus)

Un Raspberry Pi 5 a ete integre en cours de projet pour valider la compatibilite domotique du cahier des charges : integration Ollama native dans HAOS, pipeline vocal Assist (STT -> agent Ollama -> TTS). Cette brique n'est plus dans le setup actuel mais demontre l'extensibilite de la stack vers des usages domotiques et constitue un element bonus de la soutenance.

3. Identifiants et controle d'accès

Service	Port	Utilisateur	Mot de passe
OpenWebUI	:3000	ps.enzo.maresia@gmail.com	AI_serv
Flask (Plagiat)	:5000	admin	admin (a changer)
Home Assistant	172.17.0.x	rt3-raspberry	Etudiants89100
OS serveur	SSH	enzo / AI_serv	—

Cle API HuggingFace (invalide — a revoquer avant soutenance) :

```
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
```

Note : les points de securite a corriger sont detailles en section 8.

4. Phase 0 — Environnement logiciel et deployment

L'environnement est volontairement **leger et headless** : aucune interface graphique cote serveur, services IA isoles dans des conteneurs Docker.

4.1 Checklist de deployment

- [OK] Retirer les barrettes RAM defectueuses
- [OK] Installer Linux Mint 22.3 (cle USB bootable)
- [OK] Activer SSH et securiser les connexions
- [OK] Installer Docker Engine (via paquet .deb)
- [OK] Installer Ollama avec support GPU
- [OK] Charger les modeles IA
- [OK] Deployer OpenWebUI (deux instances Admin / User)
- [OK] Configurer Python 3.12.3 et le venv
- [OK] Deployer le pipeline de detection de plagiat
- [OK] Optimiser la VRAM (sentence_transformers -> CPU, LLaVA desactive)
- [EN COURS] Reinstaller les drivers GPU NVIDIA pour la soutenance

4.2 Prerequis systeme

```
sudo apt update

sudo apt install -y \

tesseract-ocr tesseract-ocr-fra tesseract-ocr-eng \

libpango-1.0-0 libpangoft2-1.0-0 \

python3 python3-venv python3-pip git curl
```

4.3 Installation de Docker Engine

L'installation via le depot APT standard a rencontre des problemes sur le serveur. La solution retenue (seance 4) a ete l'installation via le **paquet .deb** telecharge directement depuis docs.docker.com/engine/install/ubuntu.

```
sudo apt install -y ca-certificates curl gnupg

sudo install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg \

| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

# Linux Mint : utiliser le nom de code Ubuntu de base

echo "deb [arch=$(dpkg --print-architecture) \
```



```
signed-by=/etc/apt/keyrings/docker.gpg] \  
https://download.docker.com/linux/ubuntu \  
$(. /etc/os-release && echo "$UBUNTU_CODENAME") stable" \  
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null  
  
sudo apt update  
  
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin  
  
sudo usermod -aG docker $USER  
  
docker run --rm hello-world
```

4.4 Installation et configuration d'Ollama

```
curl -fsSL https://ollama.com/install.sh | sh  
  
systemctl status ollama  
  
# Exposer Ollama sur le reseau (acces depuis les postes clients)  
  
sudo systemctl edit ollama  
  
# Ajouter dans le bloc [Service] :  
  
# Environment="OLLAMA_HOST=0.0.0.0:11434"  
  
sudo systemctl daemon-reload && sudo systemctl restart ollama
```

4.5 Modeles IA installes

```
ollama pull gemma3:4b  
  
ollama pull qwen2.5:7b  
  
ollama pull llama3.2:3b  
  
ollama pull llama3.1:8b  
  
ollama pull llava:7b  
  
ollama list
```

Modele	Type	Role	Taille
gemma3:4b	Texte (LLM)	Verdict IA local — etape 6	~3 Go
qwen2.5:7b	Texte (LLM)	Assistant nomme Qwen (OpenWebUI)	~4.7 Go

Modele	Type	Role	Taille
llama3.2:3b	Texte (LLM)	Modele leger pour tests	~2 Go
llama3.1:8b	Texte (LLM)	Second avis IA local (provider)	~4.7 Go
llava:7b	Vision (VLM)	Installe — DESACTIVE (VRAM)	~4.7 Go
multilingual-e5-base	Embeddings	Similarite semantique FR/EN — CPU	~1.1 Go

4.6 Contrainte VRAM et optimisations appliquees

La RTX 3060 Ti dispose de **8 Go de VRAM**. Deux problemes ont ete rencontres en seance 8 lors de l'execution simultanee du pipeline :

Probleme 1 — sentence_transformers sur GPU : par default, la bibliotheque chargeait multilingual-e5-base sur le GPU, consommant **~2.3 Go de VRAM**. Avec Ollama qui charge egalement son modele texte, Ollama detectait l'etat limited-vram et tentait d'offloader LLaVA sur CPU.

Probleme 2 — LLaVA sur CPU et boucle CLIP : une fois LLaVA offloade sur CPU, le processus llama-server (encodeur CLIP de LLaVA) se mettait a boucler a **1100 % d'utilisation CPU**, bloquant le pipeline entier.

Solutions appliquees :

```
# Forcer sentence_transformers sur CPU dans le module d'embeddings

from sentence_transformers import SentenceTransformer

model = SentenceTransformer("intfloat/multilingual-e5-base", device="cpu")

# Desactiver completement LLaVA dans le pipeline

vision_model = None
```

Resultat : VRAM disponible pour Ollama **~7.3 Go / 8 Go**. L'etape d'analyse d'images repose desormais uniquement sur le **hash perceptuel** (bibliotheque imagehash).

4.7 Solutions rejetees

Solution	Raison du rejet
LM Studio	Le Xeon E5-2630 v3 ne supporte pas AVX2 (jeu d'instructions requis par LM Studio pour l'inference CPU)
LiteLLM	Installe mais la liaison Ollama <-> LiteLLM n'a jamais fonctionne ; abandonne au profit de l'API Ollama directe
VNC	Ports SSH/VNC bloques par l'infrastructure reseau de l'IUT — acces distant non resolu

4.8 Deploiement OpenWebUI

```
# Instance OpenWebUI — port 3000
```

```
docker run -d --name openwebui -p 3000:8080 \  
--add-host=host.docker.internal:host-gateway \  
-e OLLAMA_BASE_URL=http://host.docker.internal:11434 \  
-v openwebui:/app/backend/data \  
--restart unless-stopped \  
ghcr.io/open-webui/open-webui:main
```

L'instance dispose de son propre volume Docker — le premier compte cree devient automatiquement administrateur.

5. Phase 1 — Pipeline de detection de plagiat (CLI)

Coeur du systeme, **100 % local**. Orchestre par la fonction **analyze()** dans main.py. Emplacement : /home/enzo/Telechargements/SAE204_scripts/

5.1 Les 8 etapes de l'analyse

#	Etape	Technologie	Poids
1	Extraction PDF -> Markdown	PyMuPDF + Tesseract OCR (FR+EN)	—
2	Exclusion du sujet	Shingling — filtre les phrases de l'enonce	—
3	Detection lexicale	Shingling k=5 + rapidfuzz	35 %
4	Detection semantique	multilingual-e5-base (CPU) + cosinus	35 %
5	Analyse d'images	Hash perceptuel (imagehash) — vision LLM desactivee	15 %
6	Verdict IA local	gemma3:4b — JSON structure via Ollama	15 %
7	Scoring & risque	Agregation ponderee des 4 signaux	—
8	Generation du rapport	Markdown -> HTML -> PDF (WeasyPrint)	—

Note — Etape 5 : l'analyse vision via llava.7b a ete **desactivee** (contrainte VRAM). Seul le **hash perceptuel** est actif (distance de hash <= 10).

5.2 Parametres cles

Signal	Parametre	Valeur
Lexical	Taille des n-grammes (shingling)	k = 5
Semantique	Seuil de similarite cosinus	>= 0.85
Images	Distance de hash perceptuel	<= 10
Verdict IA	Modele utilise	gemma3:4b via Ollama

5.3 Grille des niveaux de risque

Niveau	Score global	Interpretation
[FAIBLE]	< 40 %	Aucun plagiat detecte
[MODERE]	40 % - 69 %	Similitudes a verifier manuellement
[ELEVE]	>= 70 %	Plagiat probable — intervention recommandee

5.4 Approche multi-signaux — argument soutenance

Le score global combine **quatre signaux independants** pour resister aux tentatives de contournement classiques :

- **Lexical** : Detecte le copier-coller direct et les reformulations superficielles.
- **Semantique** : Detecte la paraphrase — mots differents, sens identique. Signal absent de la plupart des detecteurs classiques.
- **Images** : Detecte la reutilisation de schemas et captures d'ecran, souvent ignoree meme par des outils professionnels.
- **Verdict IA** : Apporte un jugement contextuel avec justification en langage naturel, pondere par l'indice de confiance du modele.

5.5 Utilisation en ligne de commande

```
cd /home/enzo/Telechargements/SAE204_scripts

source venv/bin/activate

# Analyse standard (avec exclusion de l'enonce)

python3 main.py \

--a rapport_a.pdf \

--b rapport_b.pdf \

--sujet sujet.pdf \

--output resultats/

# Avec second avis via Ollama local (llama3.1:8b)

python3 main.py \

--a rapport_a.pdf \

--b rapport_b.pdf \

--external \

--provider ollama
```

6. Phase 2 — Application web Flask

Interface graphique permettant aux enseignants de **deposer les TPs** et de **suivre l'analyse en temps reel**, sans ligne de commande. Acces : <http://localhost:5000> — Fichier principal : app.py (~480 lignes)

6.1 Structure du projet Flask

```
scripts/

■■■ app.py # Flask : auth, upload, SSE, SQLite

■■■ templates/ # 8 vues HTML Jinja2

■ ■■■ login.html

■ ■■■ upload.html

■ ■■■ analyse_status.html # terminal live (SSE)

■ ■■■ ...

■■■ modules -> ../SAE204_scripts/modules # lien symbolique

■■■ main.py -> ../SAE204_scripts/main.py # lien symbolique

■■■ external_verify.py # module second avis IA

■■■ test_kimi_api.py # test API externe

■■■ venv/
```

6.2 Base de donnees SQLite

Fichier : ~/plagiat_app/plagiat.db

Table	Role
users	Comptes : login, hash SHA-256 du mot de passe, role
token_counters	Tokens consommés par utilisateur (local / externe)
analyses	Historique : dates, scores, verdict, chemins des rapports

6.3 Suivi en temps reel — SSE + monkey-patching

La page analyse_status.html affiche un **terminal simule qui defile en direct**. Fonctionnement : (1) le pipeline CLI emet des logs via des print() — (2) Flask **intercepte la sortie standard** (monkey-patching de sys.stdout) — (3) chaque ligne est **poussee au navigateur** via Server-Sent Events (SSE). Un terminal actif = pipeline operationnel. Le message "Mode Demo" signale un lien symbolique casse ou une dependance manquante.

6.4 Mode demo (fallback automatique)

Si les symlinks sont brisés ou si une dépendance est absente, l'application bascule automatiquement en Mode Demo (resultats fictifs). Reparation :

```
ln -sf /home/enzo/Telechargements/SAE204_scripts/modules \
/home/enzo/Telechargements/scripts/modules

ln -sf /home/enzo/Telechargements/SAE204_scripts/main.py \
/home/enzo/Telechargements/scripts/main.py
```

6.5 Demarrage de l'application

```
cd /home/enzo/Telechargements/scripts

source venv/bin/activate

pkill -f "python app.py" 2>/dev/null # libere le port 5000

python app.py
```

Important : toujours activer le venv avant de lancer python app.py. Sans le venv, les imports échouent et l'application passe en mode demo.

7. Verification externe — Etat actuel

7.1 Situation actuelle

La verification externe via **HuggingFace / Kimi K2.6** a ete abandonnee en seance 8 : la cle API est devenue **invalide (erreur 401)**. Plutot que de la renouveler, la decision a ete prise de remplacer cet appel distant par un **second modele local** (llama3.1:8b via Ollama). Le pipeline est desormais **100 % local**, sans aucune dependance reseau externe. C'est un renforcement de l'approche "confidentialite d'abord" du projet.

La verification externe reste desactivee dans l'application Flask. Le module external_verify.py conserve le support du provider ollama local.

7.2 Parametres API HuggingFace (reference historique)

Parametre	Valeur
Endpoint	https://router.huggingface.co/novita/v3/openai/chat/completions
Modele	moonshotai/Kimi-K2.6:fastest
Cle	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 (invalide — 401)
Format	JSON — API OpenAI-compatible

7.3 Fallback local — llama3.1:8b

Champ retourne	Signification
verdict_final	Verdict consolide par le second modele
accord_avec_local	Confirmation du verdict de gemma3:4b
confiance	Indice de confiance du second avis
justification	Explication en langage naturel

7.4 Arbitrage local vs externe

Critere	Local (Ollama)	Externe (Kimi K2.6)
Confidentialite	Aucune donnee ne sort du serveur	Extraits envoyes a un tiers
RGPD	Traitement local — conforme	A verifier selon fournisseur
Puissance	Limitee par la VRAM (8 Go)	Modele massif, meilleur raisonnement
Disponibilite	Sans Internet	Requiert connexion + quota actif
Cout	Gratuit	Credits HuggingFace

8. Securite et conformite

8.1 Approche "local d'abord"

- Prompts, copies d'eleves et rapports ne quittent jamais le serveur.
- Le second avis IA est fourni par llama3.1:8b en local.
- La verification externe est desactivee dans l'application Flask.

8.2 Points a corriger avant soutenance

Priorite	Probleme	Remediation
[CRITIQUE]	Cle HF codee en dur dans test_kimi_api.py et app.py	Deplacer dans .env, revoquer la cle actuelle
[CRITIQUE]	Compte Flask admin/admin par default	Changer via script SQLite (voir ci-dessous)
[MODERE]	Meme mot de passe AI_serv sur OpenWebUI Admin et User	Differencier par service
[MODERE]	Serveur Werkzeug en frontal (dev)	Remplacer par gunicorn pour la soutenance

8.3 Changer le mot de passe admin Flask

```
python3 -c "  
  
import sqlite3, hashlib  
  
db = sqlite3.connect('/home/enzo/.plagiat_app/plagiat.db')  
  
pw = hashlib.sha256(b'NouveauMotDePasse').hexdigest()  
  
db.execute(  
    'UPDATE users SET password=? WHERE username=?',  
    (pw, 'admin')  
)  
  
db.commit()  
  
print('Mot de passe mis a jour')  
"
```

8.4 Commandes de diagnostic

```
# Verifier les liens symboliques (si casses -> mode demo)
```

```
ls -l /home/enzo/Telechargements/scripts/modules

ls -l /home/enzo/Telechargements/scripts/main.py

# Verifier les modeles Ollama presents

ollama list

# Tester les imports Python depuis le venv

cd /home/enzo/Telechargements/scripts

source venv/bin/activate

python -c "import flask; from main import analyze; print('Imports OK!)"

# Verifier la VRAM disponible

nvidia-smi
```

8.5 Problemes frequents et solutions

Symptome	Cause probable	Solution
Mode demo affiche	Symlinks casses ou venv non active	Recreer les symlinks (§6.4), activer le venv
Port 5000 occupe	Ancien processus app.py actif	<code>pkill -f "python app.py"</code>
ImportError sur main	Symlink main.py casse	Recreer le lien symbolique
Pipeline bloque 1100% CPU	LLaVA charge sur CPU (CLIP)	Verifier que <code>vision_model = None</code> est actif
GPU non detecte	Drivers NVIDIA absents ou corrompus	<code>sudo ubuntu-drivers autoinstall</code>
Erreur 401 externe	Cle HF invalide	Utiliser le provider ollama local

9. Deroulement du projet — Seances de travail

#	Date	Objectif	Realisations cles
1	12/05	Prise en main	Depot GitHub, OS, SSH, 1er script plagiat, WebUI port 3000
2	21/05	HAOS + Ollama	HAOS active, Ollama -> HAOS via integration native, test qwen2.5:3b
3	22/05	Installation serveur	RAM defectueuses retirees, Linux Mint installe, choix qwen2.5:7b
4	27/05	Deploiement	Docker via .deb, LiteLLM, Ollama, OpenCode installes
5	03/06	Stabilisation	Consolidation de l'environnement — memes objectifs que S4
6	04/06	Mise au point IA	LMStudio abandonne (AVX2), Ollama GPU, OpenWebUI multiuser [OK]
7	05/06	Tests approfondis	Infrastructure et detecteur testes (2 versions)
8	09/06	Optimisation VRAM	sentence_transformers CPU, LLaVA desactive, provider Ollama local
9	16/06	Tests finaux	Tokens, integrite IA, securisation infrastructure, detecteur renforce

Decisions cles issues des seances :

Seance 3 : Decision de prioriser l'automatisation (plagiat) et de mettre de cote la gestion fine des tokens/logs pour une phase ulterieure.

Seance 6 : Abandon de LM Studio (pas d'AVX2 sur Xeon E5-2630 v3) et de LiteLLM (liaison Ollama non fonctionnelle). Confirmation de la stack finale : Ollama + OpenWebUI.

Seance 8 : Abandon de la verification externe HuggingFace (cle invalide 401). Remplacement par llama3.1:8b en local — pipeline 100 % local.

10. Inventaire des fichiers de projet

Les fichiers de travail sont stockes localement dans **F:\sae_projintel**.

Fichier / Dossier	Description
COMPTE_RENDU_SAE204_...md	Rapport complet (ce document — version Markdown source)
BILAN_MODIFICATIONS.md	Historique des correctifs GPU / VRAM / symlinks
plagiat_detector.py	Script CLI standalone — version initiale (531 lignes)
pyvenv.cfg	Configuration de l'environnement virtuel Python 3.12
Plan projet	Notes de conception initiales (architecture, methodologie)
api token	Cle HuggingFace hf_KKfM... — A REVOQUER avant soutenance
ligne_de_commande	Historique de 476 commandes shell saisies au cours du projet
install_openwebui_...	Log terminal complet (neofetch, netstat, Docker, NVIDIA)
image_proj_intel/	37 captures d'ecran du deploiement (reference visuelle)
projet8inte8vid/	Scripts, 3 videos de demo (.mkv), captures supplementaires

Le fichier "api token" contient la cle HuggingFace en clair. Il doit etre supprime ou chiffre avant tout partage ou depot sur un depot public.

11. Feuille de route — Phase 3

11.1 Securite (prioritaire)

- [] Changer le mot de passe admin Flask (§8.3).
- [] Deplacer HF_API_KEY dans un fichier .env (exclu du depot Git via .gitignore).
- [] Revoquer la cle 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 sur HuggingFace.
- [] Supprimer ou chiffrer le fichier "api token" dans F:\sae_projinte\.
- [] Differencier les mots de passe entre OpenWebUI Admin et User.

11.2 Ameliorations techniques

- [] Comptage des tokens : extraire prompt_eval_count + eval_count depuis les reponses Ollama et les enregistrer dans token_counters.
- [] Serveur de production : remplacer Werkzeug par gunicorn avec worker gevent pour stabiliser le streaming SSE lors de la soutenance.
- [] Calibrage des seuils : ajuster les parametres lexicaux (k-shingles) et semantiques (cosinus 0.85) sur un vrai jeu de donnees de TP.
- [] Reinstaller les drivers GPU NVIDIA correctement pour une demo fluide.

11.3 Preparation de la soutenance

- [] Jeu de test demonstratif : 1 sujet + 2 rapports dont 1 plagie (paraphrase + image reutilisee) pour illustrer les 4 signaux.
- [] Demo bout en bout : OpenWebUI (assistant nomme Qwen) -> app Flask (upload -> terminal SSE -> rapport PDF).
- [] Argumentaire "local d'abord" : confidentialite / RGPD vs puissance externe ; contrainte VRAM geree par des decisions techniques documentees.
- [] Mention HAOS : breve presentation de l'integration domotique comme preuve d'extensibilite de la stack Ollama.

12. Glossaire

Terme	Definition
Ollama	Serveur local qui telecharge et execute des LLM, exposes via une API REST (port 11434).
OpenWebUI	Interface web de chat connectee a Ollama, gestion multi-comptes (type ChatGPT).
Embeddings	Representation vectorielle d'un texte permettant de mesurer la similarite de sens (distance cosinus).
Shingling	Decoupage d'un texte en sequences de k mots consecutifs pour comparer les chevauchements lexicaux.
Hash perceptuel	Empreinte d'image robuste aux petites modifications — detecte les images reutilisees.
SSE	Server-Sent Events — canal HTTP unidirectionnel serveur -> navigateur, utilise pour le terminal live.
Monkey-patching	Remplacement dynamique d'une fonction a l'execution (ici, capture de sys.stdout).
VLM	Vision-Language Model — modele capable d'analyser des images (ex. LLaVA). Desactive dans ce projet.
Headless	Mode sans interface graphique, pilote exclusivement a distance via reseau.
VRAM	Memoire embarquee sur le GPU, limite le nombre et la taille des modeles chargeables simultanement.
AVX2	Jeu d'instructions CPU requis par LM Studio. Absent sur le Xeon E5-2630 v3 — raison du rejet.
WSGI	Interface serveur Python de production (gunicorn) remplaçant le serveur de developpement Werkzeug.

13. Recapitulatif des ports et chemins

Element	Valeur
Ollama (API REST)	http://localhost:11434
OpenWebUI	http://localhost:3000
Application Flask	http://localhost:5000
Pipeline CLI	/home/enzo/Telechargements/SAE204_scripts/
Application Flask (dir)	/home/enzo/Telechargements/scripts/
Base SQLite	~/plagiat_app/plagiat.db
Fichiers de travail	F:\sae_projinte\
IP du serveur	172.17.0.x (DHCP — reseau prive salle reseaux IUT)

Compte-rendu redigé a partir des notes de projet, du journal des seances et du depot GitHub AYK0-prog/Local_AI_server — Mis a jour le 17 Juin 2026.